

Section 8

Task 8

MULTIPLEXERS AND SHIFTERS

22-07-2026

PLAYGROUND LINK: <https://www.edaplayground.com/x/SwqT>

GIT LINK: https://github.com/Nooorrr66/MEDS_2026/tree/main/module_4/8_Multiplexers_Shifters

CODE:**Test Bench**

```

module task8_tb;
logic a, b, c;
logic F;
// Instantiate the design
task8 uut (
    .a(a),
    .b(b),
    .c(c),
    .F(F)
);

logic expected;
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, task8_tb);
    $display(" a b c | F | Expected | Result");
    $display("-----");

    for (int i = 0; i < 8; i++) begin
        // Apply inputs
        {a, b, c} = i;
        #10;
        // Expected output from Σm(1,2,3,6,7)
        expected = (i == 1 || i == 2 || i == 3 || i == 6 || i == 7);

        if (F == expected)
            $display(" %b %b %b | %b | %b | PASS",
                a, b, c, F, expected);
        else
            $display(" %b %b %b | %b | %b | FAIL",
                a, b, c, F, expected);
    end
    $finish;
end
endmodule

```

Design:

```

//top mod
module task8(
    input logic a,
    input logic b,
    input logic c,
    output logic F
);

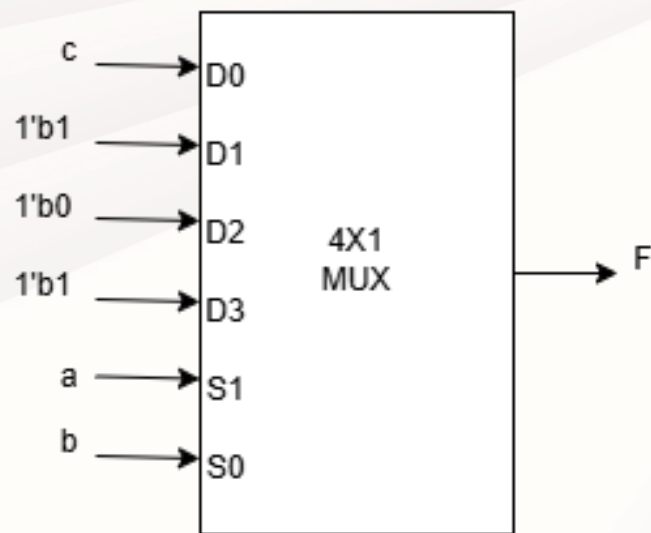
mux4to1 U1 (
    .s1(a),
    .s0(b),
    .d0(c),
    .d1(1'b1),
    .d2(1'b0),
    .d3(1'b1),
    .y(F)
);
endmodule

//design
module mux4to1(
    input logic s1,
    input logic s0,
    input logic d0,
    input logic d1,
    input logic d2,
    input logic d3,
    output logic y
);

always_comb begin
    case ({s1, s0})
        2'b00: y = d0;
        2'b01: y = d1;
        2'b10: y = d2;
        2'b11: y = d3;
        default: y = 1'b0;
    endcase
end
endmodule

```

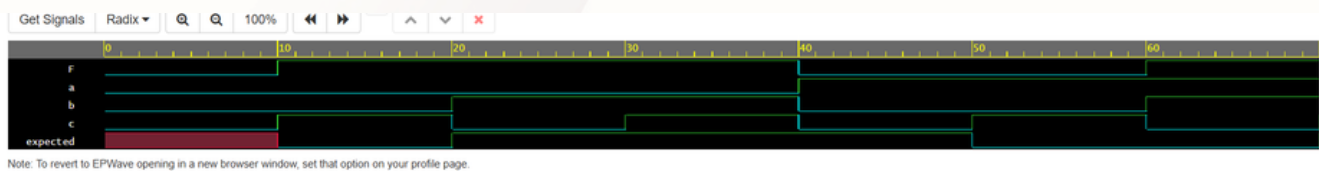
SCHEMATIC:



TERMINAL:

```
Contains Synopsys proprietary information.
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64;  Jul 22 03:19 2026
a b c | F | Expected | Result
-----
0 0 0 | 0 | 0 | PASS
0 0 1 | 1 | 1 | PASS
0 1 0 | 1 | 1 | PASS
0 1 1 | 1 | 1 | PASS
1 0 0 | 0 | 0 | PASS
1 0 1 | 0 | 0 | PASS
1 1 0 | 1 | 1 | PASS
1 1 1 | 1 | 1 | PASS
$finish called from file "testbench.sv", line 36.
```

WAVEFORM:



EXTENSION:

- Extension (include in the same report): repeat the exercise for a 3-bit majority function (output is 1 when 2 or more of a, b, c are 1), this time using only cascaded 2-to-1 multiplexers instead of a single 4:1 mux. Include the majority function's truth table and a schematic showing how the 2:1 muxes are cascaded.

PLAYGROUND LINK:

<https://www.edaplayground.com/x/YWAB>

CODE:

Test Bench

```
module majority_mux_tb;
logic a, b, c;
logic majority;
logic expected;
majority_mux uut(
    .a(a),
    .b(b),
    .c(c),
    .majority(majority)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, majority_mux_tb);
end
initial begin
    $display(" a b c | Out | Exp | Result");
    $display("-----");
    for (int i = 0; i < 8; i++) begin
        {a,b,c} = i;
        #10;
        expected = (a & b) | (a & c) | (b & c);
        if (majority == expected)
            $display(" %b %b %b | %b | %b | PASS",
                a,b,c,majority,expected);
        else
            $display(" %b %b %b | %b | %b | FAIL",
                a,b,c,majority,expected);
    end
    #10;
    $finish;
end
endmodule
```

TERMINAL:

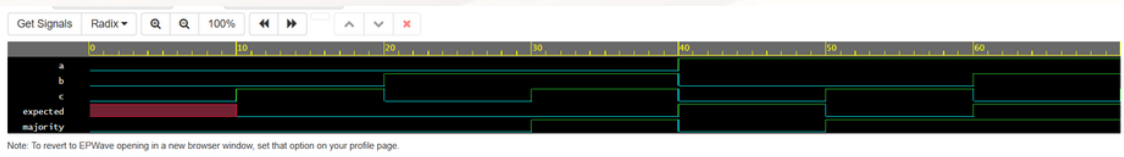
```
Compiler version X-2025.06-SP1_Full64; Runtime
a b c | Out | Exp | Result
-----
0 0 0 | 0 | 0 | PASS
0 0 1 | 0 | 0 | PASS
0 1 0 | 0 | 0 | PASS
0 1 1 | 1 | 1 | PASS
1 0 0 | 0 | 0 | PASS
1 0 1 | 1 | 1 | PASS
1 1 0 | 1 | 1 | PASS
1 1 1 | 1 | 1 | PASS
$finish called from file "testbench.sv", line 1
```

Design:

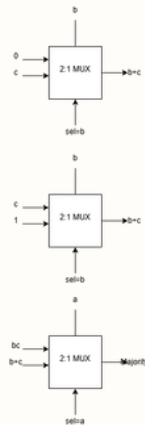
```
module mux2to1(
    input logic s,
    input logic d0,
    input logic d1,
    output logic y
);
always_comb begin
    case (s)
        1'b0: y = d0;
        1'b1: y = d1;
    endcase
end
endmodule

module majority_mux(
    input logic a,
    input logic b,
    input logic c,
    output logic majority
);
logic bc;
logic borc;
// MUX1: bc = b ? c : 0
mux2to1 M1(
    .s(b),
    .d0(1'b0),
    .d1(c),
    .y(bc)
);
// MUX2: borc = b ? 1 : c
mux2to1 M2(
    .s(b),
    .d0(c),
    .d1(1'b1),
    .y(borc)
);
// MUX3: majority = a ? (b+c) : (bc)
mux2to1 M3(
    .s(a),
    .d0(bc),
    .d1(borc),
    .y(majority)
);
endmodule
```

WAVEFORM:



SCHEMATIC



EXPLANATION:

A complete truth table was first constructed by marking the output as 1 for minterms 1, 2, 3, 6, and 7, and 0 for the remaining combinations. The variables a and b were chosen as the select inputs of the 4×1 multiplexer. For each value of ab , the required data input was determined by examining how the output changed with c .

The derived data inputs are:

- $D_0 = c$
- $D_1 = 1$
- $D_2 = 0$
- $D_3 = 1$

Thus, whenever the select lines are 00, the multiplexer forwards c to the output. When the select lines are 01, the output is permanently 1. For 10, the output is permanently 0, and for 11, the output is again permanently 1. This implements the required Boolean function using only a single 4×1 multiplexer.

Task 8. $F = \sum m(1, 2, 3, 6, 7)$ using a 4×1 MUX

Date: _____

Minterm	a	b	c	dec	F
m_0	0	0	0	0	0
m_1	0	0	1	1	1
m_2	0	1	0	2	1
m_3	0	1	1	3	1
m_4	1	0	0	4	0
m_5	1	0	1	5	0
m_6	1	1	0	6	1
m_7	1	1	1	7	1

defining MUX inputs using a 4-L as sel

a	b	c=0	c=1	data in
0	0	0	1	$D_0 = c$
0	1	1	1	$D_1 = 1$
1	0	0	0	$D_2 = 0$
1	1	1	1	$D_3 = 1$

Select Lines $= S_1 = a; S_0 = b$

The design consists of two modules.

The first module, `mux4to1`, implements a generic 4×1 multiplexer using an `always_comb` block and a case statement. The concatenated select inputs `{s1,s0}` determine which of the four data inputs (`d0–d3`) is connected to the output `y`.

The second module, `task8`, instantiates the multiplexer and connects the derived inputs:

- `s1 = a`
- `s0 = b`
- `d0 = c`
- `d1 = 1'b1`
- `d2 = 1'b0`
- `d3 = 1'b1`

The output of the multiplexer is assigned to the Boolean function `F`

The waveform confirms that:

- `F = 1` for input combinations 001, 010, 011, 110, and 111.
- `F = 0` for 000, 100, and 101.

This exactly matches the given minterm specification, verifying the correctness of the design.

The extension requires implementing the 3-bit majority function using only cascaded 2×1 multiplexers.

The majority function produces an output of 1 whenever two or more of the three inputs are equal to 1.

Using Shannon expansion with `a` as the select variable:

- When `a = 0`, the function reduces to `b·c`.
- When `a = 1`, the function reduces to `b + c`.

The design therefore uses three multiplexers:

1. The first 2×1 multiplexer generates `b AND c`.
2. The second 2×1 multiplexer generates `b OR c`.
3. The third 2×1 multiplexer selects between these two intermediate results using `a` as the select signal.

This satisfies the requirement of implementing the function using only cascaded 2×1 multiplexers.

Extension:

a	b	c	dec	#1s	Major	When a=1
0	0	0	0	0	0	b c output
0	0	1	1	1	0	0 0 0 $f=b+c$
0	1	0	2	1	0	0 1 1
0	1	1	3	2	1	1 0 1
1	0	0	4	1	0	1 1 1
1	0	1	5	2	1	
1	1	0	6	2	1	
1	1	1	7	3	1	

using Shannon expression When a=0

b	c	output
0	0	0
0	1	0 $f=b \cdot c$
1	0	0
1	1	1

The first module, mux2to1, implements a standard 2x1 multiplexer using a case statement. Depending on the value of the select input, either d0 or d1 is connected to the output.

The majority_mux module instantiates three copies of this multiplexer.

- MUX1 implements the AND operation by selecting between 0 and c using b as the select input.
- MUX2 implements the OR operation by selecting between c and 1 using b as the select input.
- MUX3 selects between the outputs of the first two multiplexers using a as the select signal to generate the final majority output.

Section 8

Task 9

Part_a

22-07-2026

PLAYGROUND LINK: <https://www.edaplayground.com/x/VUnc>

CODE:

Test Bench

```
module right_shifter_tb;
logic [3:0] W;
logic shift;
logic [3:0] Y;
logic k;
logic [3:0] expectedY;
logic expectedK;
right_shifter uut(
    .W(W),
    .shift(shift),
    .Y(Y),
    .k(k)
);
```

Design:

```
module right_shifter(
    input logic [3:0] W,
    input logic shift,
    output logic [3:0] Y,
    output logic k
);
assign Y[3] = shift ? 1'b0 : W[3];
assign Y[2] = shift ? W[3] : W[2];
assign Y[1] = shift ? W[2] : W[1];
assign Y[0] = shift ? W[1] : W[0];
assign k = shift ? W[0] : 1'b0;
endmodule
```

```

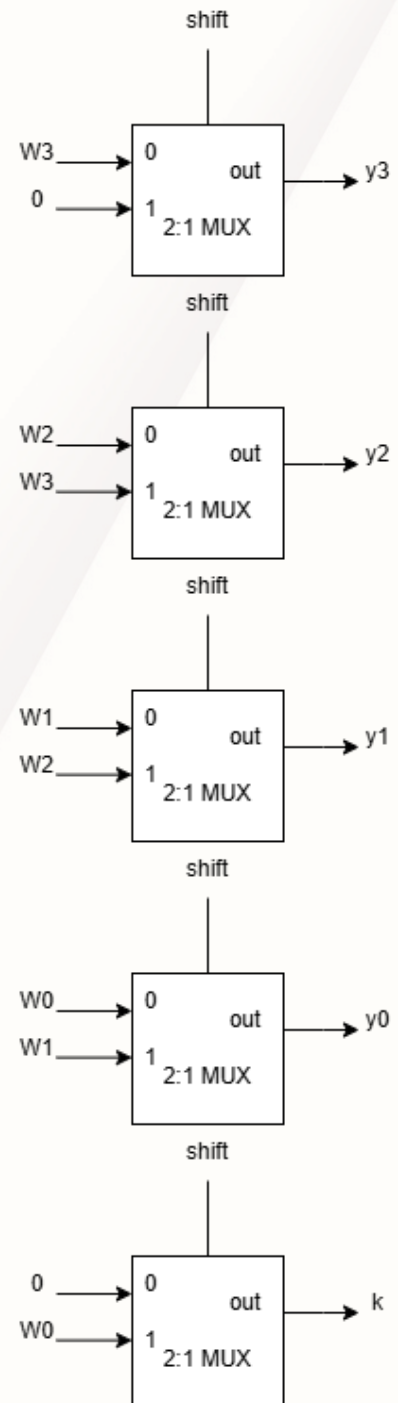
// Waveform
initial begin
$dumpfile("dump.vcd");
$dumpvars(0, right_shifter_tb);
end
initial begin
$display(" W shift | Y k | ExpectedY ExpectedK | Result");
$display("-----");
//Test 1
W = 4'b0000;
shift = 0; #10;
expectedY = 4'b0000;
expectedK = 0;
check();
shift = 1; #10;
expectedY = 4'b0000;
expectedK = 0;
check();
//Test 2
W = 4'b1011;
shift = 0; #10;
expectedY = 4'b1011;
expectedK = 0;
check();
shift = 1; #10;
expectedY = 4'b0101;
expectedK = 1;
check();
// Test 3
W = 4'b1100;
shift = 0; #10;
expectedY = 4'b1100;
expectedK = 0;
check();
shift = 1; #10;
expectedY = 4'b0110;
expectedK = 0;
check();
// Test 4
W = 4'b0101;
shift = 0; #10;
expectedY = 4'b0101;
expectedK = 0;
check();
shift = 1; #10;
expectedY = 4'b0010;
expectedK = 1;
check();
$finish;
end
task check;
begin
if ((Y == expectedY) && (k == expectedK))
$display("%b %b | %b %b | %b %b | PASS",
W, shift, Y, k, expectedY, expectedK);

else
$display("%b %b | %b %b | %b %b | FAIL",
W, shift, Y, k, expectedY, expectedK);

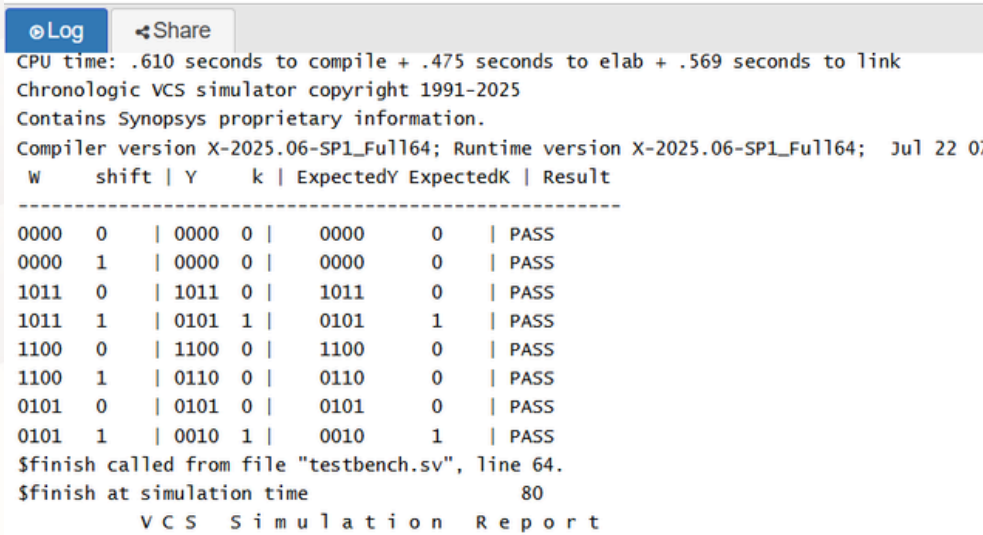
end
endtask
endmodule

```

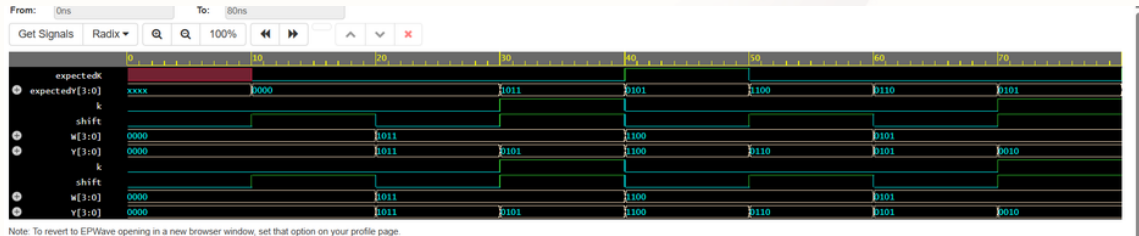
SCHEMATIC:



TERMINAL:



WAVEFORM:



Task 9
Part_b

22-07-2026

PLAYGROUND LINK: <https://www.edaplayground.com/x/GK79>

CODE:

Test Bench

```
module barrel_shifter_tb;
    logic [3:0] W;
    logic [1:0] sel;
    logic [3:0] Y;
    logic [3:0] expected;
    integer i;
    // Instantiate DUT
    barrel_shifter uut (
        .W(W),
        .sel(sel),
        .Y(Y)
    );
    // Waveform dump
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, barrel_shifter_tb);
    end
    initial begin
```

Design:

```
module barrel_shifter(
    input logic [3:0] W,
    input logic [1:0] sel,
    output logic [3:0] Y
);
    always_comb begin
        unique case(sel)
            2'b00: Y = W;
            2'b01:
                Y = {W[0], W[3], W[2], W[1]};
            2'b10:
                Y = {W[1], W[0], W[3], W[2]};
            2'b11:
                Y = {W[2], W[1], W[0], W[3]};
            default:
                Y = 4'b0000;
        endcase
    end
endmodule
```



```

$display("=====
");
$display(" Barrel Shifter Testbench");

$display("=====
");
// Test 1
test(4'b1011);
// Test 2
test(4'b0101);
// Test 3
test(4'b1100);

$display("=====
");
$display(" Simulation Complete");

$display("=====
");
#10;
$finish;
end
task test(input logic [3:0] value);
begin
W = value;
for (i = 0; i < 4; i = i + 1) begin
sel = i[1:0];
#10;
case (sel)
2'b00: expected = value;
2'b01: expected = {value[0], value[3], value[2], value[1]};
2'b10: expected = {value[1], value[0], value[3], value[2]};
2'b11: expected = {value[2], value[1], value[0], value[3]};
endcase
if (Y == expected) begin
$display("PASS : W=%b sel=%b Y=%b",
value, sel, Y);
end
else begin
$display("FAIL : W=%b sel=%b Y=%b Expected=%b",
value, sel, Y, expected);
end
end
endtask
endmodule

```

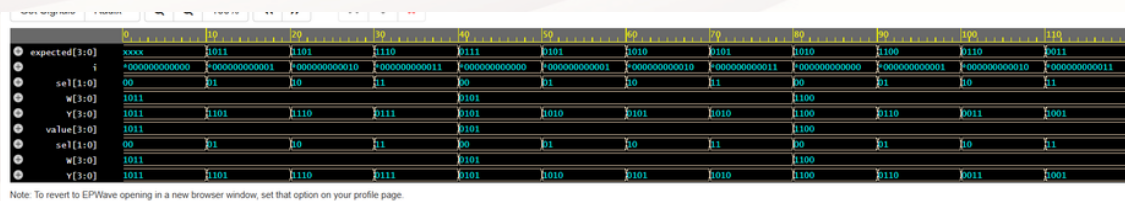
TERMINAL:

```

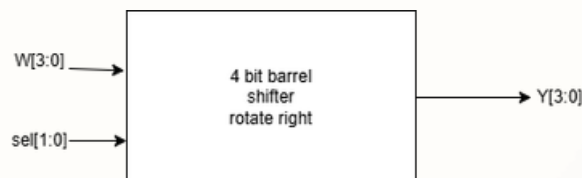
=====
Barrel Shifter Testbench
=====
PASS : W=1011 sel=00 Y=1011
PASS : W=1011 sel=01 Y=1101
PASS : W=1011 sel=10 Y=1110
PASS : W=1011 sel=11 Y=0111
PASS : W=0101 sel=00 Y=0101
PASS : W=0101 sel=01 Y=1010
PASS : W=0101 sel=10 Y=0101
PASS : W=0101 sel=11 Y=1010
PASS : W=1100 sel=00 Y=1100
PASS : W=1100 sel=01 Y=0110
PASS : W=1100 sel=10 Y=0011
PASS : W=1100 sel=11 Y=1001
=====
Simulation Complete
=====
$finish called from file "testbench.v" line 45

```

WAVEFORM:



SCHEMATIC:



EXPLANATION:

Part A implements a 4-bit single-position right shifter. The circuit accepts a 4-bit input vector $W = w_3w_2w_1w_0$ and a control signal shift. When shift = 0, the output remains unchanged ($Y = W$) and the shifted-out bit k is 0. When shift = 1, the input is shifted one position to the right. A 0 is inserted into the most significant bit, and the least significant bit (w_0) is shifted out and appears at output k .

The design is implemented using conditional (ternary) operators, where each output bit behaves like a 2:1 multiplexer controlled by the shift signal.

The module uses five conditional assignments:

```
assign Y[3] = shift ? 1'b0 : W[3];
assign Y[2] = shift ? W[3] : W[2];
assign Y[1] = shift ? W[2] : W[1];
assign Y[0] = shift ? W[1] : W[0];
assign k = shift ? W[0] : 1'b0;
```

Each assignment selects between two possible values depending on shift, which directly models a 2:1 multiplexer. No case statement is used, satisfying the task requirement.

Part B implements a 4-bit barrel shifter that performs a circular right rotation. Unlike the single-position shifter, no bits are lost during rotation. Instead, the bit shifted out from the least significant position re-enters at the most significant position.

The amount of rotation is selected using a 2-bit control signal sel.

selOperation

00 ->No rotation

01 -> Rotate right by 1

10 ->Rotate right by 2

11->Rotate right by 3

The implementation uses a unique case statement to select the required rotation.

The unique case statement informs both the simulator and synthesis tool that exactly one case item should match for every valid input.

Compared to a normal case statement, unique case provides two benefits:

- During simulation, it can detect and warn if no case or multiple cases match, helping identify design errors.
- During synthesis, it enables additional optimizations because the synthesis tool knows the cases are mutually exclusive.

Thus, unique case improves both verification and synthesis quality.